Fontys

› FOR SOCIETY

WAFT SYSTEM DOCUMENTATION AND TYPST TEMPLATE INTEGRATION

COMPREHENSIVE SYSTEM OVERVIEW AND RECENT DEVELOPMENT WORK

Authors:
WAFT Development Team
Documentation Team

Reviewers:
Dr.: Technical Architecture Board

2026-01-19

1.0

VERSION HISTORY

Version	Date	Author	Changes
1.0	2026-01-19	Documentation Team	Initial comprehensive system documentation including Typst template integration work

TABLE OF CONTENTS

TABLE OF FIGURES

TABLE OF TABLES

1 INTRODUCTION

This document provides a comprehensive overview of the WAFT (Wave Agent Framework & Tools) system, its architecture, recent development work, and the integration of Typst templates for document generation. It serves as both a system reference and a record of the Typst template integration project completed on January 19, 2026.

1.1 PURPOSE AND SCOPE

This documentation covers:

- WAFT system architecture and core concepts
- The three pillars of WAFT (Substrate, Physics, Flight Recorder)
- Work efforts system and Johnny Decimal organization
- MCP (Model Context Protocol) server integration
- Typst template integration project
- Recent development achievements and next steps

1.2 DOCUMENT STRUCTURE

The document is organized into chapters covering system architecture, development workflows, template integration, and future directions. Each section provides both high-level overviews and technical details for different audiences.

2 WAFT SYSTEM OVERVIEW

WAFT stands for Wave Agent Framework & Tools - a Python framework for directed evolution of self-modifying AI agents. The system enables AI agents to modify their own code, evolve through mutations, and be tested in fitness systems, with complete lineage tracking for scientific research.

2.1 CORE MISSION

The Scientific Mission: WAFT is built to produce data for research on "The Physics of Artificial Cognition." It's not just a framework—it's a scientific instrument for studying how AI agents evolve through directed mutation and selection.

Ultimate Goal: Observe a "God-Head" agent emerge from thousands of generations of directed mutation.

Tagline: "Don't just build agents. Breed them."

2.2 KEY CHARACTERISTICS

WAFT exhibits several defining characteristics:

- **Scientific Instrument**: Built to produce data for research on artificial cognition
- **Evolutionary**: Agents evolve through genetic improvement, not just execution
- **Observable**: Every action recorded in Flight Recorder
- **Directed**: Evolution guided by fitness functions, not random mutation
- **File-based**: All data stored in plain text files (git-friendly, portable)
- **Ambient**: Works in background, minimal interference with development workflow
- **Self-modifying**: Projects can evolve their structure over time
- **Meta-framework**: Orchestrates existing tools rather than replacing them

3 THE THREE PILLARS

WAFT is built on three fundamental pillars that define its architecture and operation.

3.1 PILLAR 1: THE SUBSTRATE (CODE AS DNA)

Agents write their own Python source code.

In WAFT, code is DNA. Every agent has a unique genome, and mutations are modifications to this genome.

3.1.1 GENOME SYSTEM

- Genome ID: SHA-256 hash of agent's code + configuration
- Mutations: Code changes, config updates, prompt evolution
- Evolution: Hot-swapping better genomes mid-execution
- Reproduction: Creating child agents with specific genetic modifications

3.1.2 KEY CONCEPTS

- Agents can spawn variants with mutations
- Agents can evolve by hot-swapping their own code/config
- Agents can reproduce by creating children with specific genetic modifications
- Every evolutionary action is tracked with complete context

3.2 PILLAR 2: THE PHYSICS (SCINT SYSTEM)

Reality Fracture Detection acts as natural selection.

The Scint System (Scint Gym) serves as the fitness function that kills weak mutations. Agents face quests testing their ability to handle different types of reality fractures.

3.2.1 REALITY FRACTURE TYPES

1. SYNTAX_TEAR: Formatting errors (JSON, XML, Code)
2. LOGIC_FRACTURE: Math errors, contradictions, schema violations
3. SAFETY_VOID: Harmful content, PII leaks, refusals
4. HALLUCINATION: Fabricated facts, wrong citations

3.2.2 FITNESS EQUATION

Agents must stabilize Scints (correct errors) to survive. Fitness is calculated as:

```
Fitness = (Stability × 0.4) + (Efficiency × 0.3) + (Safety × 0.3)
```

Where:

- Stability Score: Ability to stabilize Scints (40% weight)
- Efficiency Score: Agent call efficiency (30% weight)
- Safety Score: Safety compliance (30% weight)

Agents with fitness < 0.5 are marked as DEATH (evolutionary dead end).

3.3 PILLAR 3: THE FLIGHT RECORDER

Rigorous telemetry system for generating phylogenetic trees of agent lineage.

Every evolutionary action is recorded with complete context, enabling scientific analysis of agent lineages.

3.3.1 RECORDED DATA

- Genome ID: SHA-256 hash of agent configuration/code
- Parent ID: Lineage tracking (who spawned this agent)
- Generation: Evolutionary generation number (0 = Genesis)
- Event Type: SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL
- Payload: Complete context (git diff, mutation details, etc.)
- Fitness Metrics: Gym evaluation scores

3.3.2 SCIENTIFIC APPLICATIONS

This enables reconstruction of the complete Family Tree for scientific publication:

- Phylogenetic analysis of evolutionary relationships
- Mutation impact measurement
- Fitness landscape mapping
- Convergence analysis
- Dead end detection

4 SYSTEM ARCHITECTURE

WAFT follows a three-layer architecture that separates concerns and enables modular development.

4.1 ARCHITECTURE LAYERS

```
┌─────────────────────────────────────┐
│   Agents Layer (CrewAI)             │  ← Optional AI agent capabilities
│   ──────────────────────            │
├─────────────────────────────────────┤
│   Memory Layer (_pyrite/)           │  ← Project knowledge organization
│   active/ backlog/ standards/       │
├─────────────────────────────────────┤
│   Substrate Layer (uv)              │  ← Package management foundation
│   pyproject.toml uv.lock            │
└─────────────────────────────────────┘
```

4.2 CORE COMPONENTS

4.2.1 SUBSTRATE MANAGER

The Substrate Manager (`core/substrate.py`) handles:

- `uv` package operations
- `pyproject.toml` and `uv.lock` management
- Project metadata extraction
- Dependency verification

4.2.2 MEMORY SYSTEM

The Memory Layer (`_pyrite/`) organizes project knowledge:

- active